Chapter 1 — The AI/ML Performance Architecture Mindset

> *"Performance engineering is not about making one function faster. It is about understanding the whole system — from transistors to tokens — and knowing exactly where the bottleneck lives before you touch anything."*

Chapter Overview

This chapter is the analytical skeleton on which every other chapter in this book hangs its flesh. It introduces three quantities — arithmetic intensity, memory bandwidth, and communication volume — that are sufficient to reason about the performance of any AI/ML system at any scale. Every optimization technique in the 17 chapters that follow reduces, when analyzed rigorously, to one of these three quantities or a tradeoff among them.

Sections in this chapter:

- 1.0 Chapter Manifesto: Why This Chapter Exists
- 1.1 The Seven-Layer Performance Stack
- 1.2 Roofline Analysis: The Universal Performance Model
- 1.3 Arithmetic Intensity — The Master Classifier
- 1.4 The Three Performance Regimes
- 1.5 Hardware Efficiency: MFU and HFU
- 1.6 Measurement Discipline: How to Profile Before You Optimize
- 1.7 The Three Questions Every Chapter Answers
- 1.8 Key Takeaways and Review Questions

1.0 Chapter Manifesto: Why This Chapter Exists

Most books on GPU performance begin with hardware specs. This one begins with a mindset shift — because the specs change every 18 months, but the mindset is what makes you dangerous in year one, year five, and year ten.

Here is the shift:

Traditional systems engineering asks: *Is this system fast?* You measure latency, measure throughput, tune the bottleneck, ship the fix.

AI/ML systems engineering asks: *Why, arithmetically, is this system as fast or slow as it is — and what does the hardware's physics tell me about how much room I have left?*

This is not a philosophical distinction. It is an operational one. When a senior engineer sees an underperforming GEMM, they reach for a profiler. When a principal engineer sees the same GEMM, they first ask: what is the arithmetic intensity of this operation at this matrix shape, what is the ridge point of this GPU, and which side of that ridge point are we on? The profiler confirms a hypothesis; it does not generate one. This chapter teaches you to generate the hypothesis.

The One Sentence That Captures This Entire Book

> **"Every optimization decision in AI/ML infrastructure is a question of arithmetic intensity, memory bandwidth, and communication volume — master these three, and you can reason about any system."**

This sentence is not a simplification. It is a compression. Every technique in this book — FlashAttention kernel tiling, KV cache paging, tensor parallelism topology, gradient bucketing, quantization bit-width selection, speculative decoding batch sizing, NVLink vs. InfiniBand collective strategy — reduces, when analyzed rigorously, to one of these three quantities or a tradeoff among them. The chapters that follow are the expansion. This sentence is the zip file.

When you encounter a new system, a new hardware generation, a new paper claiming 3× speedup, or an interviewer asking you to design a 1,024-GPU training cluster, come back to this sentence. Ask: what is the arithmetic intensity of the dominant operation? What memory bandwidth is required to sustain it? What communication volume does the parallelism strategy impose? The answers locate you on the roofline. Everything else is engineering from that position.

The Mindset Shift in Practice

Traditional Systems Mindset	AI/ML Principal Mindset
"Let me profile and see what's slow."	"Let me classify the regime first — compute-bound or memory-bound — then profile to confirm."
"This kernel is at 82% GPU utilization."	"82% utilization means nothing without arithmetic intensity. What's the FLOP/byte ratio?"

Traditional Systems Mindset	AI/ML Principal Mindset
"We need more bandwidth."	"How many bytes per FLOP does this operation require, and what bandwidth is needed to keep the ALUs fed?"
"The AllReduce is slow."	"What is the message size, the topology, the algorithm (ring vs. tree), and the effective bandwidth as a fraction of theoretical?"
"We're bottlenecked on memory."	"Which memory tier — registers, L1, L2, HBM — and at what access pattern?"
"Add another node."	"What is the communication-to-computation ratio at this scale, and does adding a node help or hurt?"

The right column is not harder. It is more precise. Precision is what separates the engineer who fixes the symptom from the engineer who reasons about the system.

1.1 The Seven-Layer Performance Stack

AI/ML systems are optimized across seven distinct layers simultaneously. The performance of any real system is determined by the weakest layer at any given moment — and the weakest layer changes depending on workload, scale, and hardware configuration. A principal-level performance architect operates across all seven.

```
LAYER 7 — CLUSTER & DATACENTER
  Cluster topology, multi-rack networking, cooling capacity, power delivery,
  job scheduling, multi-tenant isolation.
  Bottleneck signals: cluster MFU < 50%, gang scheduling failures, power capping.

LAYER 6 — DISTRIBUTED SYSTEM
  Collective communication (AllReduce, AllGather, ReduceScatter, All-to-All),
  parallelism strategy (TP/PP/DP/SP), pipeline scheduling, gradient
synchronization.
  Bottleneck signals: AllReduce > 15% of step time, pipeline bubble > 10%.

LAYER 5 — RUNTIME & FRAMEWORK
  PyTorch/JAX execution graph, CUDA Graphs, operator dispatch, kernel launch
  overhead, Python GIL interaction, memory allocator behavior.
  Bottleneck signals: GPU idle > 20% between kernels, high CPU-to-GPU dispatch
latency.

LAYER 4 — KERNEL & COMPILER
  CUDA/ROCm kernels, tensor core utilization, operator fusion, tiling strategies,
  register allocation, shared memory bank conflicts.
  Bottleneck signals: arithmetic intensity below ridge point, low tensor core
activity.

LAYER 3 — MEMORY SUBSYSTEM
  HBM bandwidth utilization, cache hit rates (L1/L2), register spill, memory
  access coalescing, prefetching.
  Bottleneck signals: DRAM_ACTIVE > 80% while SM utilization low.

LAYER 2 — COMPUTE SILICON
  SM occupancy, warp scheduling, Tensor Core throughput, FP8/BF16/FP16
utilization,
  clock frequency, thermal throttling.
  Bottleneck signals: Tensor Core utilization < 60%, thermal throttle events.

LAYER 1 — PHYSICAL SUBSTRATE
  Power delivery, cooling, PCB signal integrity, NVLink/PCIe physical bandwidth,
  HBM die temperature, ECC error rates.
  Bottleneck signals: ECC corrected errors rising, GPU temperature > 83°C.
```

The Principal Architect's Diagnostic Rule: Always start at Layer 7 and drill down to the layer where the bottleneck lives. The most common mistake is starting at Layer 4 (kernel profiling) before establishing that the bottleneck is actually in a kernel, rather than in communication

overhead, scheduling, or data loading. The profiling methodology in Section 1.6 enforces this top-down discipline.

The Layer Interaction Rule: Fixing one layer without understanding its interaction with adjacent layers routinely creates new bottlenecks. Increasing tensor core utilization at Layer 4 (kernel) may shift the bottleneck to HBM bandwidth at Layer 3. Improving AllReduce efficiency at Layer 6 may shift the bottleneck to PCIe bandwidth at Layer 1. The roofline model in Section 1.2 is the tool for predicting these interactions before running experiments.

1.2 Roofline Analysis: The Universal Performance Model

The Roofline Model is the most consistently useful analytical framework in AI performance engineering. It classifies every operation into one of three performance regimes and quantifies exactly how far observed performance is from the theoretical hardware limit. Before profiling a kernel, before opening Nsight Compute, roofline analysis tells you which resource to optimize.

1.2.1 The Roofline Formula

```
ROOFLINE MODEL FUNDAMENTALS:

  Arithmetic Intensity (AI) = FLOPs executed / Bytes of memory traffic
                                  (FLOP/byte — the master classification ratio)

  Achievable performance ceiling:
    T_achieved <= min( Peak_FLOPS,  AI x Peak_Memory_BW )
                         ^                   ^
                  Compute roof        Memory roof (slope)

  At the ridge point, both ceilings are equal:
    Peak_FLOPS = AI_ridge x Peak_Memory_BW

  Therefore:
    AI_ridge = Peak_FLOPS / Peak_Memory_BW    [units: FLOP/byte]
```

This formula has two implications that every performance engineer must internalize:

1. If `AI < AI_ridge`: adding more compute (FLOPS) gives you nothing. The GPU is waiting for data. The optimization target is memory bandwidth or data movement reduction.
2. If `AI > AI_ridge`: adding more memory bandwidth gives you nothing. The compute units are the constraint. The optimization target is arithmetic throughput.

Getting the wrong answer to "which side of the ridge am I on?" means every engineering hour spent on the wrong optimization is wasted. The roofline gives you this answer in 60 seconds from first principles.

1.2.2 H100 SXM5 Ridge Point Derivation

```
H100 SXM5 ROOFLINE PARAMETERS: [SHIPPED]

  Peak BF16 FLOPS (Tensor Core, dense):   989 TFLOPS
  Peak HBM3 Bandwidth:                    3.35 TB/s

  Ridge point calculation:
    AI_ridge = 989 x 10^12 FLOP/s / 3.35 x 10^12 bytes/s
    AI_ridge = ~295 FLOP/byte

  Interpretation:
    AI < 295  -> memory-bandwidth bound
                 Performance scales with memory BW, not compute.
                 Adding FLOPS delivers zero benefit.
                 Solution: reduce data movement, fuse kernels, quantize.

    AI > 295  -> compute bound
                 Performance scales with FLOPS, not memory BW.
                 Adding HBM bandwidth delivers zero benefit.
                 Solution: increase arithmetic throughput, tile for Tensor Cores,
FP8.

  Comparison across GPU generations:
    GPU           Peak BF16      HBM BW       Ridge Point
    A100 SXM4     312 TFLOPS     2.0 TB/s     ~156 FLOP/byte   [SHIPPED]
    H100 SXM5     989 TFLOPS     3.35 TB/s    ~295 FLOP/byte   [SHIPPED]
    H200 SXM5     989 TFLOPS     4.8 TB/s     ~206 FLOP/byte   [SHIPPED]
    MI300X        1,307 TFLOPS   5.3 TB/s     ~247 FLOP/byte   [SHIPPED]
    B200 SXM      4,500 TFLOPS   8.0 TB/s     ~563 FLOP/byte   [ANNOUNCED]

  KEY INSIGHT: H200 has a LOWER ridge point than H100 despite same compute.
  More HBM bandwidth means more operations become memory-bound.
  Quantization (reducing bytes/param) becomes MORE impactful on H200.
```

1.2.3 The Roofline Plot — H100 SXM5

```
                    ROOFLINE — H100 SXM5 (BF16, Dense)
```

```
Throughput
(TFLOPS)
1000 |                                    ___________________________
     |                               ____/ COMPUTE CEILING (989T)
 989 |_____________________________/
     |              MEMORY ROOF   /|
     |              (slope =     / |
     |               3.35 TB/s) /  |<--- Ridge Point: ~295 FLOP/byte
 700 |                         /   |
 500 |                        /    |
 300 |                       /     |
 100 |                   ___/      |
  50 |                ___/         |
  20 |            ___/             |
  10 |        ___/                 |
   5 |    ___/                     |
   1 |__/                          |
     +----+----+----+-----+--------+---------> AI (FLOP/byte)
          1    5   30   100  200  295   500

Operations plotted:
  @ Decode (batch=1):      AI ~ 1     -> ~3.35 TFLOPS  ( 0.3% of peak)
  @ RMSNorm/LayerNorm:     AI ~ 5     -> ~16.7 TFLOPS  ( 1.7% of peak)
  @ GEMV (batch=32):       AI ~ 32    -> ~107 TFLOPS   (10.8% of peak)
  @ FlashAttn (S=2K):      AI ~ 200   -> ~670 TFLOPS   (68% of peak)
  @ GEMM (M=N=K=4K):       AI ~ 1365  -> ~989 TFLOPS   (fully compute-bound)
```

> **Reading the chart:** A point on the memory roof slope means the operation is memory-bandwidth-limited — its performance is determined by the slope (memory BW), not the flat ceiling (compute). A point on the flat ceiling means compute is the constraint. The gap between any plotted point and the ceiling it lies below is the optimization headroom. This gap is your budget. Everything in this book is a technique for spending that budget wisely.

1.3 Arithmetic Intensity — The Master Classifier

Arithmetic intensity is the single most important number you can know about any operation in an AI system. It determines which hardware resource limits performance, which optimizations apply, and what the theoretical maximum throughput is.

1.3.1 How to Calculate Arithmetic Intensity

```
ARITHMETIC INTENSITY FORMULA:
```

```
  AI = Total FLOPs / Total Bytes of Memory Traffic

  Memory traffic = bytes loaded + bytes stored
                   (from/to the relevant memory tier)

  Which memory tier to use:
    - For per-kernel analysis: use L2 cache traffic (Nsight Compute:
l2_global_load + store)
    - For architectural analysis: use DRAM/HBM traffic (the roofline default)
    - For system-level analysis: use total HBM traffic across all kernels
```

1.3.2 Arithmetic Intensity of Key LLM Operations

```
OPERATION ARITHMETIC INTENSITY TABLE — H100 SXM5 BF16

  Operation                    | AI (FLOP/byte) | Regime         | Notes
  ———————————————————————————————————————————————————————————————————————————
  Decode attention (batch=1)   |     ~1         | DEEPLY mem-BW  | ~295x below
ridge
  RMSNorm / LayerNorm          |     ~5         | Memory-bound   | Always a fusion
target
  Embedding lookup             |     ~1         | Memory-bound   | Table lookup
pattern
  GEMV: decode (batch=32)      |    ~32         | Memory-bound   | ~9x below ridge
  GEMV: decode (batch=128)     |   ~128         | Memory-bound   | ~2.3x below
ridge
  FlashAttention (S=2048)      |   ~200         | Near ridge     | IO-aware tiling
helps
  GEMM: prefill (batch=64)     |   ~200         | Near ridge     | Depends on seq
length
  GEMM: FFN (large batch)      |  ~300-500      | Compute-bound  | Main training
bottleneck
  GEMM: prefill (batch=256+)   |  ~400+         | Compute-bound  | Target FP8 here
  ———————————————————————————————————————————————————————————————————————————
  Ridge Point (H100 BF16)      |   ~295         | Boundary       | [SHIPPED]

  How to read this table:
  - "Memory-bound" means: more FLOPS won't help. Reduce bytes instead.
  - "Compute-bound" means: more memory BW won't help. Use FP8, better tiling.
  - "Near ridge" means: both compute and memory are partially constraining.
    Both sets of optimizations apply, but marginal returns are lower.
```

1.3.3 The Arithmetic Intensity of LLM Decode — Why Batch Size Is the Primary Lever

This is one of the most consequential calculations in LLM inference engineering. Understand it deeply.

```
LLM DECODE ARITHMETIC INTENSITY vs BATCH SIZE:

For a 70B parameter model in BF16 (2 bytes/parameter):

  Model weight bytes = 70 x 10^9 params x 2 bytes = 140 GB

  Each decode step per token:
    FLOPs  ≈ 2 x model_params x batch_size  (one forward pass = 2 FLOPs/param)
    Bytes  ≈ model_params x 2 bytes         (load all weights once per step)
           + KV_cache_bytes                  (grows with context length)

  Simplified AI at small batch, ignoring KV cache:
    AI ≈ (2 x 70B x batch_size) / (140 GB)
    AI ≈ 2 x batch_size  [FLOP/byte]

  Batch size needed to reach ridge point on H100:
    AI_ridge = 295 = 2 x batch_size
    -> batch_size_ridge ≈ 148 tokens

  In practice (including KV cache traffic):
    Effective AI at batch=1:   ~1 FLOP/byte  -> 0.3% compute utilization
    Effective AI at batch=32:  ~32 FLOP/byte -> 10.8% utilization
    Effective AI at batch=128: ~128 FLOP/byte -> deeply mem-bound still
    Effective AI at batch=256: ~256 FLOP/byte -> approaching ridge

  Maximum decode throughput at batch=1 (memory-bandwidth limited):
    tokens/sec = HBM_BW / model_bytes
               = 3.35 TB/s / 140 GB
               = ~24 tokens/sec  [ESTIMATED, ignoring KV]

  The performance engineering implication:
    At batch=1:   adding more Tensor Cores is wasted silicon.
    At batch=256: reducing weight bytes (quantization) is the path to 2x
throughput.
    Continuous batching (Chapter 6) exists specifically to increase effective
batch size.
```

> **The core insight for every serving engineer:** Batch size is the single most important tuning knob for decode throughput because it directly controls arithmetic intensity. Everything else — quantization, speculative decoding, continuous batching — is a mechanism for either increasing AI or reducing the bytes that must be loaded. Understand this and every serving optimization in this book is predictable from first principles.

1.4 The Three Performance Regimes

Every AI/ML workload at every scale lives in one of three performance regimes at any given moment. Correctly classifying the regime is the prerequisite for every optimization — the wrong fix on the wrong regime wastes engineering time and sometimes makes performance worse.

THE THREE PERFORMANCE REGIMES:

```
┌──────────────────────────────────────────────────────────────────────────┐
│  REGIME 1: COMPUTE-BOUND                                                 │
│                                                                          │
│  Definition: Tensor Cores fully saturated; HBM bandwidth underutilized      │
│  Roofline:  AI > Ridge Point (~295 FLOP/byte on H100 BF16)                 │
│  Examples:  Large-batch prefill GEMM, training forward/backward pass       │
│             at large batch, very large FFN layers                          │
│                                                                          │
│  What NOT to do: Add more HBM bandwidth — compute is the constraint        │
│  What NOT to do: Weight quantization for throughput — won't reduce BW time  │
│  What TO do:     FP8 quantization (halve FLOP cost), larger tile sizes,     │
│                  Tensor Core alignment, sparse attention                   │
│                                                                          │
│  Profiler signal: SM_ACTIVE high, DRAM_ACTIVE low, Tensor Core util > 80%  │
└──────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────┐
│  REGIME 2: MEMORY-BANDWIDTH-BOUND                                        │
│                                                                          │
│  Definition: HBM bandwidth saturated; Tensor Cores mostly idle             │
│  Roofline:  AI < Ridge Point                                               │
│  Examples:  Decode at batch=1 to ~128, LayerNorm/RMSNorm, activations,      │
│             embedding lookups, GEMV operations                            │
│                                                                          │
│  What NOT to do: Add faster Tensor Cores — compute is idle anyway          │
│  What NOT to do: Improve algorithmic FLOP count — won't affect BW time      │
│  What TO do:     Operator fusion (eliminate HBM round-trips), weight quant  │
│                  (reduce bytes loaded), increase batch size, prefix caching  │
│                                                                          │
│  Profiler signal: SM_ACTIVE high, DRAM_ACTIVE high, Tensor Core util < 20% │
└──────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────┐
│  REGIME 3: OVERHEAD-BOUND (Launch / CPU / Communication Bound)             │
│                                                                          │
│  Definition: GPU idle between work units; the bottleneck is scheduling,     │
│               communication, or Python dispatch overhead                   │
│  Examples:  Batch=1 decode without CUDA Graphs, many tiny tensor ops,       │
│             AllReduce dominating training step, data loading stalls        │
│                                                                          │
│  What NOT to do: Faster GPU or more memory — GPU is mostly idle            │
```

```
|  What NOT to do: Kernel optimization — the kernel is fine, it's the gaps   |
|  What TO do:     CUDA Graphs (capture launch overhead), torch.compile,      |
|                  overlap communication with computation, async data loading, |
|                  increase parallelism batch size to amortize overhead         |
|                                                                              |
|  Profiler signal: SM_ACTIVE low (< 40%), large gaps in Nsight Systems       |
|                   timeline, CPU utilization high during GPU idle periods     |
└──────────────────────────────────────────────────────────────────────────────┘
```

1.4.1 Quick Regime Diagnosis — 60-Second Workflow

```
QUICK REGIME DIAGNOSIS:

  Step 1: Run nvidia-smi dmon -s u for 30 seconds during workload
          -> Watch SM utilization column

  Step 2: Classify from SM utilization alone:
    SM util < 40%  during work   -> Likely Regime 3 (overhead/communication)
    SM util 40-80% during work   -> Likely Regime 2 (memory-bound) or mixed
    SM util > 80%  during work   -> Likely Regime 1 or 2 (kernel-level)

  Step 3: Open Nsight Systems (nsys profile --trace=cuda,nvtx)
    Large empty gaps between kernels? -> Confirm Regime 3
    Small gaps, continuous kernel execution? -> Regime 1 or 2, go deeper

  Step 4: If kernel-level bottleneck, open Nsight Compute Roofline tab
    Point below memory roof line? -> Regime 2 (memory-bound)
    Point at or above ridge?      -> Regime 1 (compute-bound)

  Step 5: Use the DCGM diagnostic counter table:
    SM_ACTIVE | TENSOR_ACTIVE | DRAM_ACTIVE | Diagnosis
    ──────────────────────────────────────────────────────────────────────
    High       High            Low          REGIME 1: compute-bound (healthy for
training)
    High       Low             High         REGIME 2: memory-bound (correct for
decode)
    High       Low             Low          REGIME 2 variant: elementwise ops, no
TC work
    Low        Low             Low          REGIME 3: launch overhead or data
stall
    Low        Low             High         REGIME 3 + 2: BW waste with poor
compute use
```

```
    Variable   Any             Any          Straggler: one GPU diverging from
cluster
```

1.4.2 The Regime Transition as a Function of Batch Size

The single most common performance optimization lever in LLM inference is increasing batch size. Its effect is to shift the workload from Regime 2 (memory-bound) toward Regime 1 (compute-bound) by increasing arithmetic intensity.

```
BATCH SIZE vs REGIME (70B BF16 Decode, H100):

  Batch Size  | Arithmetic Intensity | Regime            | Headroom
  ------------------------------------------------------------------------
  1           |  ~1 FLOP/byte        | Deeply mem-bound  | 295x from ridge
  8           |  ~8 FLOP/byte        | Memory-bound      | 37x from ridge
  32          |  ~32 FLOP/byte       | Memory-bound      | 9.2x from ridge
  64          |  ~64 FLOP/byte       | Memory-bound      | 4.6x from ridge
  128         |  ~128 FLOP/byte      | Memory-bound      | 2.3x from ridge
  256         |  ~256 FLOP/byte      | Near ridge        | 1.15x from ridge
  512+        |  ~512 FLOP/byte      | Compute-bound     | Above ridge
  ------------------------------------------------------------------------

  Practical implications:
  - At batch=1 to ~128: weight quantization (INT4 vs BF16) directly helps.
    Halving bytes loaded ~= doubling throughput. [ESTIMATED]
  - At batch=256+: quantization effect diminishes. FP8 GEMM helps more.
  - Continuous batching (Chapter 6) effectively raises operating batch size
    by serving multiple requests simultaneously.
```

1.5 Hardware Efficiency: MFU and HFU

Arithmetic intensity classifies a single operation. Model FLOP Utilization (MFU) measures the efficiency of the entire system — across all operations, communication, and scheduling overhead — relative to the theoretical hardware maximum. It is the single most important cluster-health metric for training.

1.5.1 MFU Definition and Formula

```
MODEL FLOP UTILIZATION (MFU):

  MFU = (Observed throughput in FLOP/s) / (Peak GPU FLOP/s)
```

```
    = (tokens_per_sec x FLOPs_per_token) / (peak_TFLOPS x 10^12 x N_GPUs)

  Where FLOPs_per_token for a transformer = 6 x N_params  (forward + backward)
        FLOPs_per_token for inference only = 2 x N_params  (forward pass only)

  Example: Training a 70B model at 1,000 tokens/sec on 8x H100 SXM5:
    FLOPs/token   = 6 x 70 x 10^9 = 420 x 10^9 FLOP/token
    MFU = (1,000 x 420 x 10^9) / (989 x 10^12 x 8)
    MFU = 420 x 10^12 / 7,912 x 10^12
    MFU = 5.3%   <- Extremely poor. Something is very wrong.

  More realistic example: 10,000 tokens/sec at same hardware:
    MFU = 4,200 x 10^12 / 7,912 x 10^12 = 53%  <- Healthy for training.

  MFU CALIBRATION REFERENCE:
    MFU < 30%:   Significant bottleneck exists. Investigate immediately.
                 Common causes: AllReduce overhead, data loading stall,
                 pipeline bubble, Regime 3 overhead at each node.
    MFU 30-45%:  Below-average. Check parallelism config, network topology,
                 gradient checkpointing overhead.
    MFU 45-60%:  Healthy range for large-scale training.
    MFU 60-70%:  Excellent. Well-tuned system on appropriate hardware.
    MFU > 70%:   Exceptional. Near production-optimal configuration.
                 Achievable on H100 with small models or specific workloads.
```

1.5.2 Hardware FLOP Utilization (HFU) — The Recomputation-Aware Variant

```
HARDWARE FLOP UTILIZATION (HFU):

  HFU is MFU's companion metric for training with gradient checkpointing.
  Gradient checkpointing re-executes forward pass activations during the backward
  pass to save memory. This is real compute the hardware is doing, but it is not
  "model progress" compute — it is infrastructure overhead.

  HFU = (Total hardware FLOPs including recomputation) / (Peak GPU FLOP/s)

  Relationship:
    HFU > MFU when gradient checkpointing is active
    The gap (HFU - MFU) / HFU = fraction of compute spent on recomputation
    Typical overhead: 20-33% recomputation with full checkpointing

  When to prefer each metric:
```

```
    Use MFU to compare training efficiency across systems and configurations.
    Use HFU to understand true hardware utilization and diagnose throttling.
    Use (MFU / HFU) to quantify the memory-compute tradeoff of checkpointing.

  Example with gradient checkpointing active:
    Measured hardware FLOP/s: 600 TFLOPS (8x H100)
    HFU = 600 / 7,912 = 7.6%  <- Low! But...
    If 30% is recomputation: effective model throughput FLOPs = 600 x 0.70
    MFU = (600 x 0.70) / 7,912 = 5.3%  <- This is what you compare externally.
```

1.5.3 Cluster MFU — The Scale-Aware Metric

When operating at scale, cluster MFU accounts for the efficiency loss from distributed communication, pipeline bubbles, and scheduling overhead that single-GPU MFU conceals.

```
CLUSTER MFU vs SINGLE-GPU MFU:

  Single-GPU MFU measures hardware efficiency on one device.
  Cluster MFU = (total cluster output in useful FLOPs) / (sum of all GPU peak
FLOPs)

  The gap between them:
    Cluster MFU = Single-GPU MFU x Scaling Efficiency

  Where:
    Scaling Efficiency = (N-GPU throughput) / (N x 1-GPU throughput) x 100%

  Typical scaling efficiency values: [ESTIMATED]
    8 GPUs (NVLink):    95-99%   (dominant loss: TP AllReduce, ~5% overhead)
    64 GPUs (1 rack):   88-95%   (adds DP AllReduce, ~5-12% overhead)
    512 GPUs:           80-90%   (adds pipeline bubbles, ~10-20% overhead)
    4,096+ GPUs:        70-85%   (large DP groups, network congestion risk)

  A cluster operating at 45% single-GPU MFU x 85% scaling efficiency =
  38% cluster MFU — the number that maps directly to $/training-token.

  Always report cluster MFU when discussing training cost or efficiency.
  Single-GPU MFU hides the communication and scheduling losses that
  dominate at scale.
```

The most common failure mode in AI performance engineering is not lack of profiling tools — it is lack of profiling discipline. Engineers run benchmarks before the GPU has reached thermal steady-state. They measure kernel time rather than system-boundary latency. They average latency numbers that have fat-tailed distributions. They run a single trial and report it as truth. Each of these errors produces misleading numbers that lead to wrong optimization decisions.

This section establishes the non-negotiable measurement rules. Apply them every time.

1.6.1 The Five Non-Negotiable Measurement Rules

```
RULE 1: WARM THE SYSTEM FIRST

  GPU performance is not constant from a cold start. GPU boost clocks ramp over
  the first 30-90 seconds of sustained load. HBM temperature rises and affects
  bandwidth (warm DRAM has higher refresh overhead). Driver JIT compilation occurs
  on first kernel launch. CUDA context initialization adds overhead to the first
  few operations.

  Protocol:
    - Run at least 5 warmup iterations before recording any measurement
    - For sustained workloads: warm for 3-5 minutes before benchmarking
    - For production capacity planning: benchmark at thermal steady-state (GPU
temp
      should be stable ± 2°C for 2 minutes before recording)
    - Verify warmup: observe GPU temperature plateau with nvidia-smi dmon -s t

  Common mistake: Cold-start benchmarks overstate throughput by 5-20%.
  The first run after a driver load will JIT-compile kernels — never include it.

RULE 2: MEASURE AT THE SYSTEM BOUNDARY

  Kernel time != user-perceived latency. The gap between them is real and large:
    - Python dispatch overhead: 50-200 μs per kernel call without CUDA Graphs
    - CUDA runtime overhead: 10-50 μs per cudaMalloc, cudaMemcpy
    - PyTorch allocator overhead: variable, can reach 500 μs in large models
    - Scheduling and synchronization: 10-100 μs per barrier

  For inference serving benchmarks: measure wall-clock time from request received
  to first token returned (TTFT) and from first token to last token (TPOT ×
length).
  For training benchmarks: measure step time including data loading, forward pass,
```

```
  backward pass, optimizer step, and AllReduce. A fast kernel inside a slow step
  is an incomplete optimization.

  Correct measurement pattern:
    torch.cuda.synchronize()          # flush all pending GPU work
    t_start = time.perf_counter()
    output = model(input)
    torch.cuda.synchronize()          # wait for GPU to complete
    t_end = time.perf_counter()
    latency = t_end - t_start

RULE 3: USE CUDA EVENTS FOR KERNEL-LEVEL TIMING

  CPU timing (time.perf_counter) includes CPU-GPU dispatch latency and is
  unsuitable for measuring individual kernels. Use CUDA events for
  sub-millisecond GPU timing:

    start_event = torch.cuda.Event(enable_timing=True)
    end_event   = torch.cuda.Event(enable_timing=True)
    start_event.record()
    <--- GPU work here --->
    end_event.record()
    torch.cuda.synchronize()
    elapsed_ms = start_event.elapsed_time(end_event)

  Accuracy: CUDA event timing is accurate to ~5 μs. Use this for kernel
  profiling. Use wall-clock timing at system boundary for E2E benchmarks.

RULE 4: REPORT PERCENTILES, NOT AVERAGES

  Latency distributions in AI serving systems are not Gaussian. They have
  long right tails driven by KV cache fragmentation, batch composition
  variability, and garbage collection. An average latency number hides
  the tail behavior that violates SLAs and drives user churn.

  Minimum reporting standard:
    P50  (median):  typical user experience
    P95:            high-percentile experience, affects ~1 in 20 requests
    P99:            SLA target for production systems (use this for commitments)
    P99.9:          tail behavior, needed for high-volume systems

  In practice: if P50 = 40ms and P99 = 400ms, the average of ~55ms tells
  you almost nothing useful about either the typical case or the failure mode.
```

```
  For throughput benchmarks: report sustained throughput over 5+ minutes,
  not peak throughput from a 30-second burst.

RULE 5: MEASURE AT THERMAL STEADY-STATE

  Modern GPUs use boost clocks that sustain above TDP ratings for short bursts,
  then throttle to base clocks under sustained load. A 30-second benchmark at
  boost clocks may show 15-25% higher throughput than the sustained production
  rate. For capacity planning, sustained thermal steady-state performance is
  the only number that matters.

  Thermal steady-state protocol:
    - Run workload for 10+ minutes before recording
    - Monitor GPU temperature: nvidia-smi dmon -s t -d 1
    - Temperature should plateau (H100 typically stabilizes at 65-75°C under load)
    - Record performance only after temperature has been stable for 2+ minutes
    - If temperature exceeds 83°C: thermal throttle is active, check cooling
```

1.6.2 The Structured Profiling Workflow — Top-Down Discipline

The profiling workflow is the operationalization of the Seven-Layer Performance Stack (Section 1.1). Always start at the system level. Only drill to the kernel level after confirming the bottleneck is in a kernel, not in communication or scheduling.

```
STEP 1: ESTABLISH THE BASELINE
  Compute MFU (training) or throughput/TTFT P99 (inference).
  Compare against expected value from roofline calculation or hardware spec.
  Gap > 20% from expectation: investigate.
  Gap < 10%: healthy, normal operational variance. Stop here.

STEP 2: CLASSIFY THE REGIME
  Is SM utilization < 50% during work? -> Regime 3 (overhead/communication)
  Is AllReduce time > 15% of step time? -> Communication bottleneck (Layer 6)
  Is data loading time > 5% of step time? -> I/O bottleneck (Layer 5)
  Are all of the above < thresholds? -> Kernel-level bottleneck (Layers 2-4)

STEP 3: OPEN NSIGHT SYSTEMS (nsys)
  Use for: system-level timeline, communication overlap, scheduling gaps
  Command: nsys profile --trace=cuda,nvtx,cudnn --output profile.qdrep python
train.py
  What to look for:
    - Large empty gaps between kernel launches? -> Overhead or Regime 3
    - NCCL kernels not overlapping with compute? -> Communication bottleneck
    - CPU-bound gaps visible in timeline? -> Python dispatch, data loading stall

STEP 4: IF KERNEL-LEVEL, OPEN NSIGHT COMPUTE (ncu)
  Use for: per-kernel hardware counter analysis, roofline position
  Command: ncu --set full --kernel-name your_kernel_name python script.py
  What to look for:
    - Roofline position: memory roof or compute roof?
    - Tensor Core utilization < 60%? -> Dtype alignment, tiling, occupancy issue
    - DRAM throughput < 60% of peak? -> Poor memory coalescing, excessive L2
misses

  IMPORTANT: Never open Nsight Compute without first running Nsight Systems.
  NCU slows execution by 100-1000x. Confirm the bottleneck is in a specific
  kernel before paying this cost.

STEP 5: VALIDATE THE FIX AT SYSTEM LEVEL
  A kernel improvement that does not improve system MFU or E2E throughput
  has not solved the real bottleneck — it has exposed the next one.
  Always validate optimizations at the same system boundary you started from.

TOOL SELECTION QUICK REFERENCE:
  "Where does step time go?"          nsys   (system timeline, ms resolution)
  "Why is THIS kernel slow?"          ncu    (hardware counters, within-kernel)
  "Is GPU healthy?"                   dcgmi dmon (continuous fleet health)
```

```
  "Is AllReduce overlapped?"          nsys   (NCCL stream visibility)
  "Is networking the bottleneck?"     nccl-tests allreduce_perf (busbw)
  "Is SM utilization correct?"        dcgmi dmon -e 1003,1004,1005
```

1.6.3 Measurement Validity: Avoiding the Common Traps

```
TRAP 1 — MICROBENCHMARK EXTRAPOLATION:
  A 40% TFLOPS improvement in a single kernel means nothing if that kernel
  is 8% of wall time. Always weight improvements by their fraction of step time.
  Real throughput gain = kernel speedup x kernel wall-time fraction.

TRAP 2 — GPU UTILIZATION AS A PROXY FOR PERFORMANCE:
  "GPU utilization at 95%" does not mean the system is performing well.
  A GPU can be 95% "utilized" while being entirely memory-bandwidth-bound
  and achieving 3% of its compute peak. Utilization measures occupancy,
  not efficiency. Use MFU and arithmetic intensity, not utilization percentage.

TRAP 3 — COMPARING RESULTS ACROSS CONFIGURATIONS WITHOUT CONTROLLING VARIABLES:
  Two benchmarks measured at different batch sizes, context lengths, or precision
  settings are measuring different workloads, not the same workload optimized.
  When comparing: hold all variables constant except the one being tested.

TRAP 4 — SINGLE-TRIAL BENCHMARKS:
  GPU performance varies run-to-run by 2-5% from thermal variation, memory
  allocator state, and OS scheduling. Report the median of at least 5 runs.
  For CI pipeline regression detection: use a rolling 7-day baseline with
  statistical significance testing (z-score or Mann-Whitney U).

TRAP 5 — BATCH EFFECTS IN DISTRIBUTED TRAINING:
  A 1.8x throughput gain in a single-node benchmark may completely disappear
  at cluster scale if the change increases gradient size or AllReduce frequency.
  Always validate distributed training optimizations at the intended scale,
  or at minimum with a scaling efficiency measurement across 1, 8, and 64 GPUs.
```

1.7 The Three Questions Every Chapter Answers

Every principal-level performance question in AI/ML infrastructure is a variant of one of these three questions. These questions appear in system design interviews, architecture reviews, and hardware procurement decisions. Every chapter in this book is an expansion of the analytical tools needed to answer them.

Question 1: Where Is the Bottleneck, and in Which Regime Does It Live?

Is this operation compute-bound, memory-bandwidth-bound, or overhead-bound? The roofline model answers this in 60 seconds from first principles. Without a regime classification, you are optimizing blind — applying bandwidth-reduction techniques to a compute-bound workload, or adding FLOPS to a memory-bound one.

Example — correctly applying the question: "Our 70B model inference is running at 12 tokens/sec per GPU at batch=1. The H100 has 3.35 TB/s HBM bandwidth. Theoretical maximum for 140 GB model at batch=1: 3,350 GB/s / 140 GB = 24 tokens/sec. We are at 50% of bandwidth limit — the bottleneck is HBM bandwidth utilization efficiency, not compute. The fix is not quantization for FLOP reduction — it is weight quantization to reduce bytes loaded per token."

Question 2: How Far Am I from the Hardware Limit?

Hardware efficiency (MFU) tells you how much performance you are leaving on the table relative to what the silicon can theoretically deliver. The gap between your measured throughput and the hardware ceiling is your optimization budget. Everything in this book is a technique for spending that budget wisely.

A system at 25% MFU has at minimum 3× performance potentially available through optimization. A system at 62% MFU has at most 1.6× available. The former warrants investment; the latter warrants careful investigation of whether the remaining gap is physically recoverable given the workload's arithmetic intensity.

Question 3: What Changes When I Scale?

A technique that works at batch size 1 on a single GPU may become bottlenecked by AllReduce at 1,024 GPUs. A memory optimization that works on an H100 SXM may behave differently on a PCIe variant with different HBM bandwidth. A CUDA Graph that eliminates launch overhead at 1 GPU may conflict with pipeline scheduling at 8-way pipeline parallelism.

This question requires the communication volume axis that roofline alone does not cover. The full framework is:

```
COMPLETE ANALYTICAL FRAMEWORK — THREE QUANTITIES:

  1. Arithmetic Intensity = FLOPs / bytes
     -> Classifies single-node compute vs memory-bandwidth bound

  2. Memory Bandwidth Demand = bytes_per_operation / time_per_operation
     -> Determines which memory tier is limiting (registers, L1, L2, HBM)

  3. Communication Volume / Compute Ratio = bytes_communicated / FLOPs_computed
     -> Determines when distributed communication becomes the bottleneck

  The scaling question: as N_GPUs increases:
    - FLOPs per GPU: stays constant (or decreases with data parallelism)
    - Communication volume: increases (DP AllReduce grows with N_GPUs)
    - At some scale, communication/compute ratio exceeds the threshold where
      AllReduce time > compute time -> scaling efficiency collapses

  For ring AllReduce (dominant algorithm for DP):
    AllReduce time = 2(N-1)/N x gradient_bytes / ring_bandwidth

  Cross-over point where AllReduce dominates:
    AllReduce_time > compute_time
    -> 2(N-1)/N x gradient_bytes / ring_BW > FLOPs/TFLOPS
    -> Solve for N to find the "scale ceiling" for a given parallelism strategy
```

These three questions recur in every chapter. Return to this manifesto whenever the detail obscures the structure. The structure is always: regime, efficiency gap, scale behavior.

1.7.1 How This Chapter Connects to Every Other Chapter

Chapter 1 is the analytical skeleton on which every other chapter builds.

Chapters 2 (Transformer Architecture): You cannot reason about transformer performance without understanding the roofline. Chapter 1 gives you the model; Chapter 2 gives you the workload — the specific matrix shapes, sequence lengths, and memory access patterns that determine arithmetic intensity for every transformer operation.

Chapters 3A, 3B (GPU/Accelerator Architecture and Roadmap): Chapter 1's abstract "memory bandwidth" and "FLOPS" become concrete when Chapter 3 shows you the physical implementation — HBM stacks, NVLink topology, Tensor Core precision modes. The ridge point formula in Section 1.2 requires the GPU spec numbers in Chapter 3.

Chapters 4–5 (Memory Hierarchy, Thermal/Power/TCO): The memory hierarchy is the physical implementation of bandwidth limits. Thermal and power constraints are the ceiling on sustained arithmetic throughput. Chapter 1's abstractions become Chapter 4 and 5's constraints.

Chapters 6–9 (Inference, Kernels, Quantization, Fusion): Every optimization technique in these chapters either increases arithmetic intensity (making a bandwidth-bound operation compute-bound) or reduces memory movement (lowering the bandwidth requirement). Chapter 1 is the lens that makes each technique legible as a category before you learn the specifics.

Chapters 10–12 (Training, KV Cache, Benchmarking): Distributed training adds the communication volume axis. Pipeline parallelism, tensor parallelism, ZeRO sharding — each is a strategy for managing the three-way tradeoff between compute efficiency, memory footprint, and communication volume that Section 1.7 introduces.

Chapters 13–17 (Speculative Decoding, Networking, Storage, Cluster, Telemetry): Performance at scale introduces communication volume as the third axis. The cluster-level MFU concept from Section 1.5 is what ties a 10,000-GPU system together as a single unit of analysis.

Chapter 18 (Principal Career): The engineer who can walk into an architecture review and say, "this design is memory-bandwidth-bound at these shapes, here is the roofline, here is the gap, here are two approaches to close it with their TCO implications" — that is the principal. Chapter 1 builds the language. Chapter 18 shows you how to use it.

1.8 Key Takeaways

```
THE ONE SENTENCE:
  "Every optimization decision in AI/ML infrastructure is a question of arithmetic
  intensity, memory bandwidth, and communication volume — master these three,
  and you can reason about any system."

ROOFLINE:
  Achievable performance <= min(Peak_FLOPS, AI x Peak_Memory_BW)
  Ridge point = Peak_FLOPS / Peak_Memory_BW
  H100 SXM5 ridge point: ~295 FLOP/byte (BF16 dense) [SHIPPED]
  Below ridge: memory-bound. Above ridge: compute-bound.

THE THREE REGIMES:
  Regime 1 (Compute-bound):   AI > ridge. Optimize arithmetic throughput.
  Regime 2 (Memory-bound):    AI < ridge. Reduce data movement.
  Regime 3 (Overhead-bound):  GPU idle. Eliminate launch/communication overhead.
  Wrong-regime optimization wastes engineering time and can worsen performance.

MFU TARGETS (training):
```

```
  < 30%:  Critical bottleneck. Investigate immediately.
  30-45%: Below average. Check parallelism, network.
  45-60%: Healthy. Well-tuned system.
  60-70%: Excellent. Near production-optimal.
  70%+:   Exceptional.

MEASUREMENT RULES — NON-NEGOTIABLE:
  1. Warm the system first (5+ warmup iterations, thermal steady-state).
  2. Measure at the system boundary, not at the kernel boundary.
  3. Use CUDA events for kernel-level timing.
  4. Report P50/P95/P99, not averages.
  5. Measure at sustained thermal steady-state for capacity planning.

PROFILING WORKFLOW — TOP-DOWN:
  Always: System MFU -> nsys timeline -> ncu kernel (only if kernel is the
bottleneck)
  Never: Open ncu before confirming kernel is the bottleneck with nsys.

THE THREE QUESTIONS:
  1. Where is the bottleneck, and in which regime?
  2. How far am I from the hardware limit?
  3. What changes when I scale?
```

Review Questions

Difficulty: Foundational

Question 1 (Ridge Point Derivation): An MI300X GPU has 1,307 TFLOPS peak BF16 performance and 5.3 TB/s HBM bandwidth. (a) Calculate the ridge point in FLOP/byte. (b) A GEMM operation at batch=64 for a 70B model achieves arithmetic intensity of approximately 100 FLOP/byte on this GPU. What regime is it in, and what is the correct optimization direction? (c) If HBM bandwidth were increased to 6.0 TB/s without changing FLOPS, how would the ridge point change, and what is the performance engineering implication?

Expected answer: (a) 1307/5.3 ≈ 246 FLOP/byte. (b) 100 < 246 → memory-bandwidth bound; optimize by reducing bytes per forward pass (quantization, KV cache compression) or increasing effective batch size, not by adding compute throughput. (c) New ridge = 1307/6.0 = 218 FLOP/byte — a lower ridge point means more operations become memory-bound; quantization becomes more valuable because more workloads are now in Regime 2 on this hardware.

Difficulty: Intermediate

Question 2 (Decode Throughput Calculation): You are deploying a 13B parameter BF16 model on an H100 SXM5. A colleague reports achieving 45 tokens/sec per GPU at batch=1. (a) What is the theoretical maximum decode throughput at batch=1 for this model on H100? (b) What MFU does 45 tokens/sec correspond to? (Use FLOPs/token = 2 × N_params) (c) The colleague suggests upgrading to FP8 to double throughput. Is this correct? Justify using the roofline framework.

Expected answer: (a) Model bytes = 13B × 2 = 26 GB. Max tokens/sec = 3,350 GB/s / 26 GB = ~129 tokens/sec theoretical (ignoring KV cache). (b) MFU = (45 × 2 × 13×10^9) / (989×10^12) = 1.18×10^12 / 989×10^12 = 0.12% — extremely low MFU because decode is memory- bound at batch=1. (c) No — FP8 reduces FLOP cost (Regime 1 fix), but the workload is in Regime 2 (AI ≈ 2 FLOP/byte at batch=1). The correct fix is to increase batch size or apply INT4 weight quantization (halving bytes loaded → approaching 2× throughput by reducing HBM traffic, not reducing FLOPs).

Difficulty: Principal Level

Question 3 (Scaling Analysis): You are designing a distributed training system for a 70B parameter model on a 512-GPU H100 cluster. Your current configuration achieves 45% cluster MFU. An engineer proposes switching to a tensor parallelism degree of TP=16 (from TP=8) to reduce activation memory. Before running the experiment, what analytical reasoning would you apply to predict whether this will improve or hurt MFU? What specific measurements would you take before and after?

Expected answer: TP=16 doubles the AllReduce communication per transformer layer because the TP AllReduce runs on twice as many GPUs. Within a single DGX H100 node (8 GPUs), TP=8 uses NVLink (900 GB/s); TP=16 must cross node boundaries using InfiniBand (typically 400 GB/s HDR200), a 2.25× bandwidth reduction. The AllReduce latency will increase by a factor of approximately 2 (more GPUs) × 2.25 (less bandwidth) = ~4.5× per AllReduce call. At 45% cluster MFU, AllReduce overhead is likely already 15-25% of step time. A 4.5× increase in AllReduce time would push that to 68-100% of step time — catastrophic for MFU. Prediction: MFU will drop significantly. The correct approach to reduce activation memory is gradient checkpointing or ZeRO stage 3, not wider TP. Measurements: (1) nsys timeline before change — measure current AllReduce fraction of step time. (2) nccl-tests allreduce_perf across 16 GPUs (spanning node boundary) to get expected AllReduce latency. (3) Compute expected new MFU analytically before running the experiment. (4) If running: measure cluster MFU, step time breakdown, and AllReduce fraction after change. Validate prediction.

Difficulty: Principal Level

Question 4 (Regime Misdiagnosis): A performance engineer reports: "Our CUDA kernel is hitting 82% GPU utilization during LLM decode, so the system is well-optimized. We see 18

tokens/sec on our 13B BF16 model at batch=1 on H100." Identify all the errors in this analysis and provide the correct interpretation.

Expected answer: Three errors. (1) "82% GPU utilization" is meaningless without classifying the performance regime. GPU utilization measures SM occupancy, not efficiency. A decode kernel can have 90% SM occupancy while achieving < 1% of compute peak, because all that "utilization" is SM warps waiting for HBM data (Regime 2). (2) 18 tokens/sec at batch=1 on H100 for a 13B BF16 model: theoretical max = 3,350/26 ≈ 129 tokens/sec. The system is at 18/129 = 14% of the memory bandwidth limit — deeply suboptimal even for a memory-bound workload. This suggests HBM bandwidth utilization is low, possibly due to non-coalesced memory access, KV cache overhead not accounted for, or scheduling overhead. The optimization is not to improve kernel FLOP efficiency — the workload is Regime 2. Fix candidates: (a) measure actual HBM bandwidth utilization with ncu to verify the coalescing pattern, (b) profile whether KV cache loading is adding substantial bytes per token, (c) check for scheduling/launch overhead with nsys. The claim "well-optimized" is incorrect — 14% of theoretical maximum is significantly underperforming.

Further Reading

1. *Williams, Patterson, Hennessy — Roofline: An Insightful Visual Performance Model for Floating Point Programs and Multiprocessor Architectures* (2009). The original roofline paper. Read the full derivation. `https://people.eecs.berkeley.edu/~kubitron/cs252/handouts/papers/RooflineVyNoYellow.pdf`
2. *Making Deep Learning Go Brrr From First Principles* — Horace He (2022). The best practical guide connecting transformer workload structure to GPU performance regimes. Required reading for every AI performance engineer. `https://horace.io/brrr_intro.html`
3. *NVIDIA H100 GPU Architecture Whitepaper* (2022). Source of truth for H100 specifications. Know the numbers on pages 8-12 cold. `https://resources.nvidia.com/en-us-tensor-core`
4. *Nsight Systems User Guide* — NVIDIA documentation. Learn the profiling methodology, not just the tool interface. `https://docs.nvidia.com/nsight-systems/`
5. *Scaling Laws for Neural Language Models* — Kaplan et al. (2020). `arxiv.org/abs/2001.08361` The quantitative framework for connecting model size, data, and compute budget. Essential background for reasoning about training efficiency at scale.

— End of Chapter 1 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved